FOSS IS fun

A Testing Time

The other day, I had an opportunity to see how software development takes place in the 'industry'—in other words, the proprietary world. I did not know whether to laugh or cry!

First, the sales team meets the client, and promises him anything whatsoever—as long as they get the order. Then the payment schedule is set up, and a hefty advance is taken. Next, the folks in design move in—they may or may not meet the client, but they produce a huge number of specs, graphs, diagrams, etc. Again, the client may or may not get to see them. Then, it's over to 'Production'. Here, the application is chopped up into bits, and handed over to various teams—in such a way that no team gets an overview of the whole application.

Soon, the coding starts. Each coder is given a little bit to do, while all coders are given commit rights. Since the left hand does not know what the right hand is doing, a lot of the code is duplicated—or the same functionality is implemented in several different ways, in the same application. At this stage, no attempt is made to check if the various parts work together. Peer review—that is, one coder criticising the work of another—is never done; it is seen as an insult. Then the pieces of code are all sent to an 'integrator', who is the most highly paid person in the project. He performs some magic to make everything work together.

The application then goes to the testing group. Testing is something everyone hates to do. If those in this department find bugs and flaws, it goes back to the integrator to be resolved. Finally, the application goes to the service and support team, who installs it for the customer. This is perhaps six months to a year after the order was placed. The customer's business needs would have changed by then, but since the money has been paid, the client silently accepts what is delivered. The customer then does some testing (a task usually assigned to the vendor's support team, who hasn't a clue as to what is happening, as the production team that originally worked on the application, has moved on, elsewhere). So the customer either accepts and makes the best of what's been delivered or ends up spending more money to fix/modify things that should not have been broken in the first place. Although this sounds crazy, apparently this is how the software industry works.

Open source development follows a totally different path. For one, the customer is pulled in as a contributor to development. The application runs from, practically, day one; the customer sees the application in all stages of development, and is encouraged to play with it by entering real data. The developers see the whole application at all times, and hence code is not duplicated, nor is the same thing done differently in different parts. Peer review of code is encouraged, everyone improves, no one takes offence. So when the customers actually get the application, they are familiar with it, and find that they've got what they wanted. And it works.

So what about testing? Testing is the fun part of development—and the developers write the tests. The mantra is: "First write your tests, then write the code. As you keep writing the code, run your tests. When all tests pass, stop writing code." Of course, it does not work in such a linear fashion. One usually writes some proof-of-concept code at first, then tests it; this is followed by more code, then more tests. When the developer is also the tester, bugs and problems tend to vanish. And when the customer is playing with the application using real data, functionality is also ensured.

Of course, to really understand and use this model, one has to view an application as an incrementally growing and evolving process—not something to be 'produced' and delivered when complete. Software is never final or complete. END

By: Kenneth Gonsalves

Kenneth Gonsalves is a FOSS activist and consultant based in Ooty, and can be reached at *lawgon@thenilgiris.com*.